# DIGITHINGS

## the digithings book

Chris Stefan · digithings.ai/book

2026 edition · version 1

# CONTENTS


01 Why this book exists ..... 3

02 Software that lives anywhere ..... 4

03 Open source, open core ..... 5

04 How it scales ..... 6

05 The scaffolding, built once ..... 7

06 Built from parts you know ..... 8

07 What you can build ..... 9

08 digigraph, the orchestration hub ..... 11

09 digillm, one model layer ..... 12

10 MCP, every capability as a tool ..... 13

11 digisearch, finding what matters ..... 14

12 digiquant, the research vertical ..... 15

13 digichat, the interface people touch ..... 16

14 digikey, identity checked everywhere ..... 17

15 digitrace, seeing what the system did ..... 18

16 The local stack, one machine ..... 19

17 Start on your machine ..... 20

01

# Why this book exists

digithings began as one builder's own AI stack: a way to use AI for real work, on a laptop or in the cloud, without a vendor deciding which model, database or interface to use. It's all published, under the MIT licence.

AI tooling changes every few months. A better model shows up, or a cheaper vector store, or a new agent client, and most applications need a rewrite to take it on.

Here, it's a setting.

## What digithings holds to

Six rules digithings builds by. Each is taken up again later, and the argument matters more than the rule.

**Own the stack.** Every part is code you can read, run and change, and when an answer looks wrong you follow it down to the line that decided. Chapter 03 makes the case.

**Location is a deployment decision.** Chapter 02 is the argument. The same services run in Docker on a laptop and in containers in a cloud account, and read the same setting names in both. Moving them changes values, not code.

**Open at the seams.** Models are reached over the OpenAI chat format, tools over MCP, identity as signed tokens, traces over OpenTelemetry. Standards the stack didn't invent, so any part can be swapped for another that speaks them. Chapter 03 argues it, and says what stays closed.

**Small parts, one job each.** Model access, orchestration, retrieval, identity, tracing and the interface are separate modules, so a project that only needs search runs retrieval and leaves the rest off. Chapter 04 turns it into three dials.

**Grow in place.** A solo builder and an organisation use the same contracts, and the scaling chapter shows how far that carries before the ceiling.

**Show the working.** Answers cite the passages they used, backtests record the engine build that produced them, and with tracing on, every request leaves a trail. The environment chapter is where those records come from.

## How to read it

The case comes first: software that can live anywhere, an open core, a stack that scales without a rewrite, and what people build with it. Then the modules, each with what it does, where its seam is and what it costs to run. It ends on your own machine, with the first thing to build.

The module chapters assume you're comfortable with Docker and a settings file. If nobody on your team is, the last chapter says who does that work.

Read it through in one sitting, or open the chapter your question lives in.

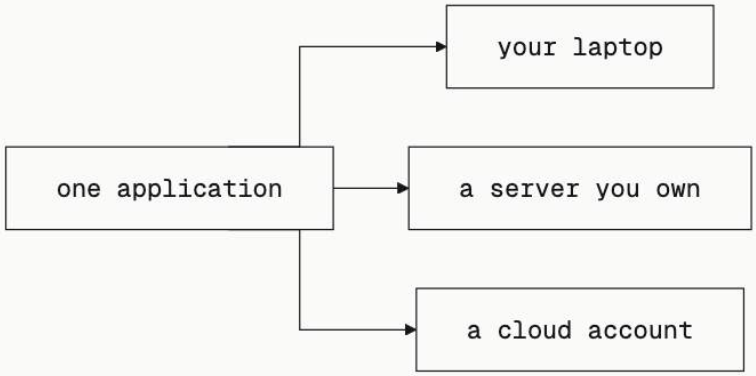
02

# Software that lives anywhere

An AI application shouldn't be tied to the machine it first ran on. Over its life the model endpoint changes, the data grows, and the one analyst who built it is joined by a team. The application shouldn't have to change with them.

An assistant product decides where your software lives on day one. You get an account, a workspace, and your data lives in their region. An agent framework built around one vendor's runtime only runs where that runtime runs. A pile of your own scripts runs exactly where you put it and nowhere else. By the time you know what the application needs, the choice has been made for you, and undoing it is a migration project.

digithings takes the opposite position. The same code runs on a laptop, on a server you own and in a cloud account, with the same service contracts, the same setting names and the same identity model in each. The services run in containers, so a move between those homes is a deployment decision rather than a rewrite. You build locally, prove the application on real data, and move it when the work needs more capacity.




one application, three homes, and the same contracts between modules in each

This isn't an argument against the cloud. A cloud account is often the right home for a production service. The argument is against choosing infrastructure before you know what the application needs, and against infrastructure that makes the choice hard to revisit. What you take on is the running. Updates, backups and the machine are yours to keep. If you'd rather not, digithings runs it for you.

Three things keep the choice open. The code is open source, so nothing is hidden behind an account. The seams between modules are standard protocols, so any module can run on a different machine from its neighbours. And no module assumes where the others live. Each finds its neighbours through configuration, so addresses stay out of the code.

So location stops being an architectural commitment. Stay on one machine while the work is small. Move to a cloud account when it needs more capacity. Put one module on its own hardware when it grows faster than the rest, say an index that outgrows the laptop's disk. At every step it is the code you started with.

03

# Open source, open core

digithings is open core. The building blocks are free under the MIT licence, and the business is what's built on top of them. Those blocks are the parts you would build an AI application from: the orchestration hub, the model layer, retrieval, backtesting and optimisation, the chat interface, identity, tracing and the shared libraries. You can read them, run them, change them and ship them inside your own product.

## Why open

Open source is usually argued on price. The stronger reasons are practical.

**You can inspect it.** When an answer looks wrong, you can follow the request through code you can read, down to the line that made the decision. A closed component turns that question into a support ticket.

**You can move it.** Portability depends on knowing every assumption a component makes about where it runs. In a closed component, those assumptions are exactly what you can't see.

**It outlives the vendor, digithings included.** Software you build on keeps working whatever happens to the company that wrote it. The code is in your hands, under a licence that doesn't expire.

## Open at the seams

Open code isn't enough if the boundaries are proprietary. A stack can publish every module and still lock you in through the way they talk to each other.

digithings keeps its seams on standards it didn't write. Models are reached through the OpenAI chat format, which Ollama, LiteLLM and most providers accept. Tools are described with the Model Context Protocol, so any client that speaks it can use them. Identity travels as signed JSON Web Tokens that any service can verify with a published key, and traces leave as OpenTelemetry. Replace a module with your own that keeps its seam and the others don't notice.

## What stays closed

The closed part is narrow and has a rule. Anything a person would reasonably want to self-host for their own work stays open. What is closed is the commercial layer built on top. That means the finance workflows that run research cycles and portfolio decisions, the layer that turns research into live orders, the machinery for running many tenants on one deployment, and managed hosting.

For a client, that is the part digithings builds and runs: research cycles on your data, the orders that follow a decision, and the deployment that keeps both going.

04

# How it scales

"Does it scale?" means one thing for a rented service and another for a stack you run. A rented service scales when a vendor raises a limit on a dashboard, and the bill follows. A stack you run scales when you decide what to add and where. That decision is yours. So is the work of running it. The stack keeps that work to what one person can carry beside another job: updates, backups, and a trace when something sounds wrong.

## Two kinds of scale

The first kind is load: more people, more requests, more documents. The chat holds no conversation state of its own, so ten copies behave like one. The hub can keep its threads in Postgres, but rate limits and pending approvals stay in the process, so a busier day means a bigger machine before it means more machines.

The second kind is capability. A chat assistant gains retrieval, then a research pipeline, then tools that act on other systems. Each step is one more module switched on.

## Three dials

**Which modules run.** Start with the hub, the model layer and the chat. Add retrieval, research, a vault or tracing when the work calls for them. A module you don't run costs nothing.

**Which models answer.** A small local model is free per call and private. A large hosted model reasons better and costs per token. The model layer routes each task, so different tasks can use different models at once.

**Where it runs.** One machine, one server, or both. Each module can sit wherever its workload is cheapest.

| Who | What they run | What grows, and the bill |
|---|---|---|
| A researcher with a laptop and a folder of papers | Hub, model layer, chat and one vertical, with local models | Disk as documents accumulate, then memory when the local model can't keep up |
| A small team sharing one corpus | Shared retrieval, a research pipeline and identity on one server or a small cloud | Memory and disk for the index, then a second machine |
| An organisation running several teams | Every module, tracing on every request, several machines or a cloud | Requests per second and stored traces, sized per module |

The rows are one system at three sizes. The contracts and setting names match in each, and the application built in the first row is the one running in the last.

05

# The scaffolding, built once

An AI application is more than a model call. It needs a way to reach models, route work between them, find the right context, call tools, identify who is asking, show an interface and record what happened. Most projects build that themselves before they reach the part that is theirs.

digithings is that scaffolding, built once. Each module has one job and a published contract, so you can run one of them, a few, or all.

| Module | Its one job |
|---|---|
| digigraph | The hub. Routes each request to the right tools and streams the answer. |
| digillm | The model layer. One way for every service to call any model. |
| digisearch | Retrieval. Indexes your documents and finds the passages that matter. |
| digivault | Documents. Linked markdown notes, readable by people and tools. |
| digiquant | Research. Data, strategies, backtests and optimisation, in order. |
| digivoice | Voice. Dictation, rewriting and speech, able to run on your own machine. |
| digichat | The interface. Where people ask, read answers and see artifacts. |
| digikey | Identity. Short-lived credentials every service checks itself. |
| digitrace | Tracing. A record of what every request did. |
| digibase | The shared base. Plumbing every service uses. |

The reference docs cover digivault, digivoice and digibase, which have no chapter here. A vertical, in these chapters, is a capability service — search, research, voice — not an industry.

The Model Context Protocol is the layer underneath, not a module: an open standard for telling a model which tools it may call. Five modules publish their tools over it, so Claude Desktop or an agent you wrote can call them, and digigraph can call any MCP server you add. That is why the table is not closed.

## How the parts fit

A request arrives in digichat or from a caller speaking the same interface, and goes to digigraph, which asks each vertical which tools it offers and runs the ones this request needs. digillm makes the model calls. digikey sits beside the flow, issuing the credential each service verifies for itself, and digitrace records the steps. Replace a part with your own that keeps its seam, and the rest carry on.

06

# Built from parts you know

A stack can look larger than it is. Strip the names away and digithings is a handful of services, each standing on projects that already have users and documentation: a graph library for the hub, an OpenAI client for model calls, a vector index for retrieval, a backtesting engine for research, a token standard for identity. Naming them is the fastest way to see how much of the stack is ordinary.

Recognition answers the first question a technical buyer asks: could their own team run it? The harder question is what sits between the parts. A library gives you a graph or an index. It does not decide which service owns the run, what a result must carry, or how a caller proves who it is. That is the work in the third column.

| Module | The open part | What digithings adds |
|---|---|---|
| digigraph | LangGraph | A tool registry the verticals publish into, a tool allowlist, one streamed answer shape |
| digillm | OpenAI client, Pydantic | Shared request and answer types, retries, a response cache, usage records and a ledger of what left for a provider |
| digisearch | Chroma, pgvector | Ingest and answer as separate paths, reranking, workspace filters, citations |
| digiquant | NautilusTrader | A fixed stage order, provenance on every result, a partial mark when data is missing, the pipeline as tools |
| digichat | OpenID Connect, Postgres | A sign-in traded for a short-lived token, history that can stay in the browser |
| digikey | JSON Web Tokens, bcrypt | One issuer, narrow scopes, fifteen-minute tokens, revocation, verification against a published key |

Two parts sit outside the table. digitrace is one helper in every service that emits OpenTelemetry, so a request keeps one trail across service boundaries and the tracing backend is a setting. The local stack is one Docker Compose file with a profile per optional module, so the core starts together and the rest arrives when the work does.

If the parts were the product, you could glue them yourself. What digithings adds is the contract each part keeps: the query index has several backends behind one interface, models are reached by name through one routing layer, sign-in accepts any OpenID Connect provider, and the backtesting engine is pinned to one exact build so a result can be reproduced. The parts are ordinary. The seams are the work.

07

# What you can build

The same modules make different applications. A use case is a choice of modules, and each starts on a single machine.

## Retrieval over your own documents

Most of what a team needs answered sits in its own files: contracts, notes, past research. Retrieval splits those documents into passages, indexes them by meaning and hands the ones that matter to the model along with the question. The answer cites what it stands on, so a reader can check it.

A workspace filter can travel with each query, so one index can hold several teams' documents. The same pipeline runs against a folder on a laptop and against a department's archive on managed storage.

## Financial research

A research desk needs market data, a way to test ideas against history, and results it can defend. Most tooling for this is either a spreadsheet that cannot be audited or a closed platform whose methodology you take on trust.

digiquant gives the work an ordered pipeline. Data, strategy validation, backtest, optimisation and export follow in turn, each stage reading only what the stage before it wrote. digigraph routes questions to it, digisearch brings in the filings and notes around a position, and digichat is where the analyst asks and reads. Every result carries its provenance, so an analyst can defend a number to someone who doubts it.

On one machine this is a private research environment for a single analyst. For a team, the pipeline moves to a shared server, backtests run on a timetable, and identity decides who can see which research.

## Voice

Speech is the input people are least willing to share, and the usual way to process it is to stream it to someone else's servers. digivoice starts from the opposite default. It captures the audio, transcribes it with Whisper, offers an optional rewrite by a small local model, and speaks the result back with Piper, all on the speaker's own Mac, which is the only platform it runs on for now. The audio is transcribed where it is spoken, and a hosted model can take its place when that's the better trade.

Because the output is ordinary text, voice becomes a way into the rest of the stack. A finished transcript pastes into whatever is in focus: a question in the chat, or a research brief dictated while your hands are busy. digichat can also switch on the browser's own dictation and read-aloud. That is convenient, but it hands the audio to the browser's engine, which in Chrome means Google's.

## Agents and exploration

An agent is a model that is allowed to act. It is given tools, such as search this index or open a ticket, and it picks one, reads the result and picks again until the task is done. It needs the tools, the loop, a way to stop and ask before anything risky, and a record. digigraph runs the loop, MCP carries the tools, a gate per tool server decides what may run unattended, and digitrace keeps the trail. The gate is checked where the tool actually runs, so a prompt that talks a model into a forbidden tool still fails at the gate.

The first agent is usually a handful of scripts a two-person ops team already runs, each given a name, an input schema and a description and served from a small MCP server. Claude Desktop or a code editor can connect to the same server, so the tools you write once work in the stack and outside it. Trying a new idea means adding one tool.

## At a glance

| Use case | Modules it runs | Starts on | Grows into |
|---|---|---|---|
| Retrieval | digisearch, digivault, digillm, digichat | A folder on a laptop | A department's archive on managed storage, with access rules per query |
| Financial research | digiquant, digigraph, digisearch, digichat | One analyst's machine | A shared research server with scheduled runs and access by role |
| Voice | digivoice, digichat | The Mac with the microphone | Any app on that machine: the transcript pastes wherever the cursor is |
| Agents | digigraph, digitrace, your MCP servers | Your own scripts as MCP tools | Many tool servers under one approval gate, shared with outside agents |

## One question, four modules

The use cases aren't separate lanes. An analyst dictates a question about a holding, and digivoice turns it into text on the speaker's own machine. In the chat, the hub asks the verticals what they offer: digisearch returns the filing passages behind the position, and digiquant reruns the backtest under the stated parameters. The answer cites both, and with tracing on, the trail shows every step that produced it.

No new code was written for that question. Each module did its one job, and the application is the set of parts that were switched on. The same chain runs unchanged when the research moves to a shared server, and that's how an idea proved on one machine grows without starting again.

08

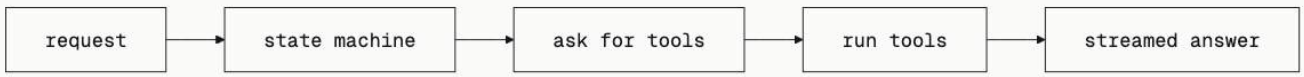
# digigraph, the orchestration hub

When a request arrives, something has to decide what happens next. A chat message, a research job and a dashboard widget all need the same loop: understand the request, pick the tools, run them in order and return the result. Without a hub, every caller writes that loop again and the copies drift apart. digigraph owns it once, so every interface shares one routing behaviour, one tool contract and one streaming format.

The hub is kept deliberately small. It routes, dispatches and streams, and leaves the real work to the verticals that own it. A hub adds one more hop on every request, and one place where a routing mistake affects every caller.

## How it works

digigraph runs each request through a state machine built on LangGraph. The first stage runs the tools the request needs. Later stages are optional and vary by use case. A question about documents ends after the first, while a trading strategy goes on through validation, backtesting and optional optimisation. Around the machine sits a registry of tools. A few live inside the hub. The verticals own the rest and publish their descriptions, the name, the inputs and what the tool does, which the hub fetches when it first needs them.




the hub asks each vertical what it offers instead of assuming it

Each service writes its own tool descriptions, so it can improve them without a hub release. A question about a company's filings reaches retrieval because retrieval described a document search, not because the hub was told where documents live.

## Where it fits

The hub speaks the OpenAI chat format, so digichat, Open WebUI or a short script can call it and read the streamed answer. Downstream it calls the verticals for retrieval and research, and models through the shared model layer. Identity arrives as a short-lived signed credential that the hub verifies itself rather than trusting the caller. With tracing on, every step is recorded, so a routing choice can be questioned after the fact.

No address is written into the hub's code, so moving a vertical to another machine changes one line of its configuration.

09

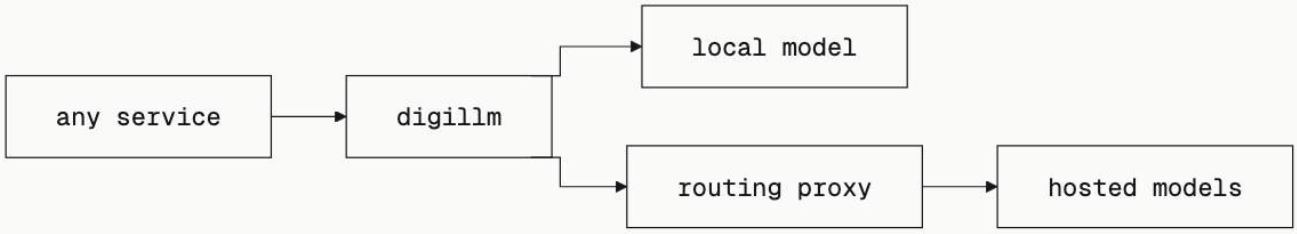
# digillm, one model layer

Left alone, every service grows its own model client. Each picks a favourite vendor, retries in its own way and fails in its own way. A year later, switching models means touching every service, and nobody can say what a model call costs. digillm exists so that never happens: one library, used by every service, with no preference for any vendor.

With a shared layer, changing models is a configuration choice. digillm speaks the OpenAI chat format, which most providers, local runners such as Ollama and routing proxies such as LiteLLM accept, so they all look the same to the calling code. Trying a new model changes where the traffic goes, not how the code asks for it. The catch is that a feature only one provider offers waits until the format can carry it.

## How it works

digillm is a library first. It ships inside the services that use it and defines the shapes every model call shares: the request, the tool call, the tool definition and structured answers checked against a schema. On top of those shapes it adds what every caller needs and nobody wants to write twice: retries, timeouts, usage accounting and optional tracing.




any service reaches any model through one layer; the provider is a setting

The library has two hard dependencies, the OpenAI client and Pydantic, and never imports a web framework. So the same code can live inside an orchestration hub, a retrieval service and a research pipeline without dragging their worlds along.

## Where it fits

The hub, retrieval and research all make their model calls through it. Because the shapes are shared, a structured answer produced in one service can be read in another without translation. When tracing is configured, usage flows into the trail; when it is not, the hooks cost nothing.

A library has no home of its own. In the local stack it points at LiteLLM, which routes each model name to Ollama on the same machine or to a hosted provider. Where models run stays an operator decision, never a code change.

10

# MCP, every capability as a tool

An AI model on its own can only write text. To do anything else, search an index, run a backtest, open a ticket, it needs tools: named functions with described inputs that it may ask to call. The Model Context Protocol (MCP), published by Anthropic in 2024 and since adopted by many AI clients, is the open standard for offering those tools. Each one has a name, a schema for its inputs and a description a model can read.

MCP is not a module, and you do not run it. It is a layer on the edge of each module that has something to offer. digigraph, digisearch, digiquant and digivault each run an MCP server beside their HTTP API, and the digillm library ships an optional one.

## Two directions

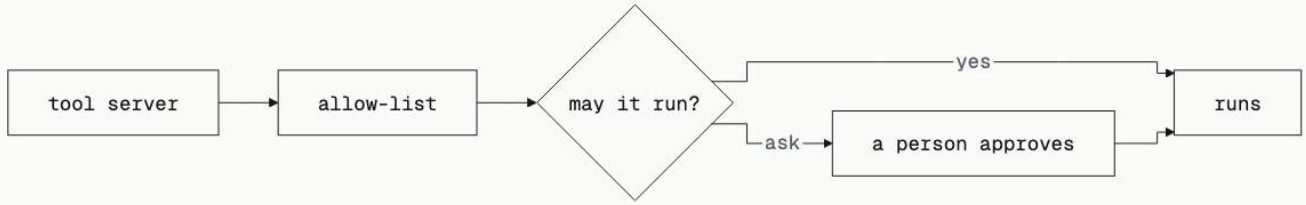
**Out.** Any MCP client can use those servers: Claude Desktop, a code editor, or a terminal agent you wrote, over a local pipe or across a network. No digichat and no hub sit in between. When a better client ships next year, it can search your index the day it ships.

**In.** digigraph is itself an MCP client. An operator lists tool servers, the team's tracker or someone else's product, and the hub offers their tools to the model on the next request, with no change to the hub.

The hub reaches its own verticals over a plainer HTTP manifest, because it treats their results specially: citations, vault notes, backtest results. The trade is that a new first-party tool needs a change in the hub, while a tool on an MCP server needs none.

## The gate

Each server the operator adds says which of its tools are allowed and which of them change something. Allowed read-only tools run unattended. A tool that changes something stops the turn and shows the exact call in the chat until whoever is at the keyboard approves or declines it.




offered and executed are checked separately, so a model only sees tools it may call

The gate is checked where the tool executes, not only where it is offered, so a prompt that talks a model into a forbidden tool still fails at the gate.

11

# digisearch, finding what matters

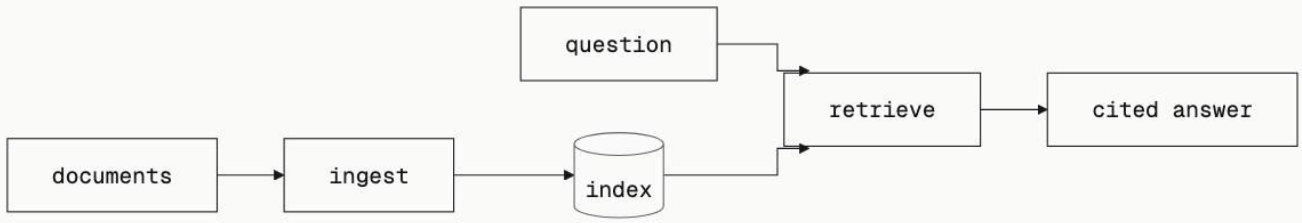
Language models know a great deal and remember nothing of yours. Everything an organisation knows, its documents, its history, its earlier research, sits outside the model. digisearch brings it in. It takes in your documents, indexes them by meaning and finds the right passages when a question needs them.

Without retrieval a model answers from training memory: fluent, confident and sometimes invented. With it, the answer arrives with the passages it stands on, and a sceptical colleague can check them. digisearch makes that the default shape of every answer that touches private knowledge.

Passages go to whichever model answers, so confidential work belongs on a local one.

## How it works

Two paths stay strictly apart. Ingestion splits documents into passages, turns each into an embedding, a list of numbers that places text by meaning, and stores them in an index, so a question about staff turnover finds a passage that only says attrition. Answering takes a question, runs a keyword search and a meaning search together, reranks the results and hands the best passages to the model with the question, so the model reasons over supplied evidence instead of recalling.




two paths share an index but never logic: indexing cannot change how a question is answered

The paths share storage and nothing else, and they need different permissions: a search key can ask questions and cannot add documents. When a workspace is set, the service adds its filter to every query, so one index can hold several workspaces and one team can't read another's documents, even by asking. The catch is that nothing is searchable until someone ingests it.

## Where it fits

digisearch sits beside digiquant as one of the hub's research arms: quant work answers from market data, search answers from documents. The chat leans on it for grounded answers and the research pipeline calls it for background. Ingestion runs on demand, and either way it lands in the index the answering path reads.

The index backend is one setting: Chroma on local disk by default, pgvector in Postgres, Cloudflare Vectorize or Azure AI Search. In every home, ingestion is an explicit act, so nothing enters the index you did not put there.

12

# digiquant, the research vertical

digiquant is one example of what a domain service looks like: one discipline's rules, packaged so the rest of the stack can call them. Here the discipline is quantitative research, and its failure mode is unusual. Results look rigorous and are not: a backtest that peeks at the future, steps run in a convenient order, a number nobody can reproduce. Each one manufactures confidence. digiquant makes those outcomes hard to produce by accident, because the honest order of operations is the only order the pipeline will run.

A digiquant number is only as good as the process behind it, so the process is fixed. Research follows one path. Each stage declares what it reads, and every result records the engine and build behind it, so it can be checked later.

## How it works

The unit of work is the whole pipeline. A caller asks for a research task; digiquant runs the stages in order on NautilusTrader, an open trading engine pinned so any result can be reproduced, and returns the result with its provenance attached.




each arrow is a contract: a stage reads only what the stage before it wrote

The engine replays history one bar at a time, so a strategy cannot read a price before its moment; that is what keeps look-ahead bias out. The fixed order keeps the rest out: nothing is optimised before it has been validated and backtested. A result whose engine cannot be identified comes back marked partial and not citable, with no plausible number to mislead anyone. The cost is one engine. A strategy written for another has to be ported.

## Where it fits

digiquant publishes its tasks as tools, to the hub and over MCP, so a caller can ask for research without knowing how quant work is ordered. Results go out to whoever asked, whether that is the chat, a scheduled session or a dashboard. The stack has one rule here: no other service may state a Sharpe ratio, a profit or a trade count unless it came from digiquant. Each result names the engine version, the pin and the lockfile it ran on, so a number on a dashboard leads back to the build behind it.

The pipeline runs wherever compute is cheapest: beside the data for fast backtests, on a timetable in the cloud, or on hardware sized to the workload. The ordering rules travel with it, because they are written into the code.

13

# digichat, the interface people touch

Most people never meet the stack. They meet digichat, the chat surface where questions go in and answers stream out. Everything behind it, the hub, the verticals and the models, serves that surface, and the surface itself does very little.

An analyst signs in, opens a conversation and asks why a holding moved. Answers stream in as they are produced, history is kept per conversation, and artifacts such as documents and visuals render beside the chat rather than inside a wall of text. The same interface can be embedded in another site, so a host can offer the experience to its own visitors with the same backend behind it.

## How it works

digichat is two parts that travel together: a request layer and a surface. The request layer signs the person in through any OpenID Connect provider, swaps that session for a short-lived digikey token and forwards the message to the hub with it. The surface renders the streaming answer, the history and the artifacts.

Neither half holds anything worth stealing. Sessions are verified on every request. Conversations live in the browser, and in Postgres when one is configured, never in the chat server itself. An embedded visitor gets a credential issued for that one visit and that one host, so registering a host grants nothing by itself.

## Where it fits

digichat relies on digikey for identity and on the hub for answers. The hub is its default backend, not its only one: one setting points it at an OpenAI-compatible endpoint, Anthropic, Google Vertex or Azure AI Foundry instead, which is useful for a plain chat with no tools behind it.

It works the other way too. The hub speaks the standard chat format, so any client that speaks that format and carries a valid token can sit where digichat sits.

## Where it runs

On a laptop it runs beside the stack it talks to, so development stays a single checkout. In the cloud it is a container in front of the hub and verticals; on your own servers, wherever you already serve web traffic.

In every home it stays stateless. Stop it, move it or run ten copies, and conversations survive, because the server never held them.

14

# digikey, identity checked everywhere

In a system of a dozen services, the dangerous question is who vouches for whom. If every service trusts every caller, one compromised caller owns everything. digikey is the narrow answer: exactly one service issues the signed credentials services accept, every other service checks them for itself, and no service ever takes a caller at its word.

Two kinds of caller need access. Services and outside systems hold long-lived access keys, shown once when issued, and exchange one for a signed token that lasts fifteen minutes. People sign in through the interface, and the request layer trades their session for the same short-lived credential. Both come from the same issuer.

## How it works

Issuing is central and checking is local. The identity service looks at what it is shown, an operator credential, a session or a machine credential, and returns a short-lived signed credential carrying identity, tenant and scopes. Every other service fetches the issuer's public signing keys, keeps them for five minutes, and checks each incoming token itself: issuer, audience, expiry and scopes. No call home is needed per request, so if the issuer goes quiet for a few minutes, requests keep working.




one service signs; every other service verifies locally with the public keys

Keys are stored scrambled, so nobody can read them back. Lose one and you get a new one. Scopes keep every credential narrow: a key scoped to search can read documents and cannot add them.

## Where it fits

Everything protected meets digikey: the hub, the verticals and the data paths. Each service declares which scopes its routes need, and shared code enforces them the same way everywhere. Outside callers come in through the same exchange, so there is no side door with different rules.

Identity lives in one service, so there is one place to issue, list and revoke keys. That makes it a single point of trust, and the design contains it. Tokens expire in minutes, scopes are narrow, and revoking a key also blocks the tokens already issued from it. It costs one more service to run. Because checking is local, identity works the same wherever the stack runs.

15

# digitrace, seeing what the system did

When a client asks why the assistant said what it said, or a regulator asks six months later, someone has to be able to show the answer's working: which tools ran, in what order, what each one saw and how long it took. digitrace is how digithings answers. It records the trail of every request as it happens, so nobody has to piece it together from log lines afterwards.

The trail serves several readers. A support engineer chasing a bad answer on a Friday afternoon follows it step by step. An auditor checking a research number reads the same trail a quarter later. And it shows where to improve: a tool that takes four seconds, or a call that fails one time in ten, stands out at once.

## How it works

digitrace has two faces. As a library, it gives every service a thin helper that emits OpenTelemetry data to whatever endpoint the deployment names. OpenTelemetry is the open standard for traces, so services depend on no tracing vendor. As a small service, it reports the tracing state to the hub and the dashboards, so whoever is on call can see it without touching sensitive configuration.

one request, one story: the trail follows it across every service it touches

The trail follows a request across service boundaries. A step in the hub and the step it triggered in a vertical appear in one story, so nobody has to match two logs by timestamp.

## Where it fits

Every service that acts carries the helper, so where a trace goes is one setting. Because the format is OpenTelemetry, switching to a different tracing backend is a configuration change.

The trail goes wherever you point it: a backend on your own machine, one beside your cloud workers, or one on your own servers with the retention you choose. Tracing every request costs some storage and a little time per request, far less than debugging a multi-service answer from scattered logs.

16

# The local stack, one machine

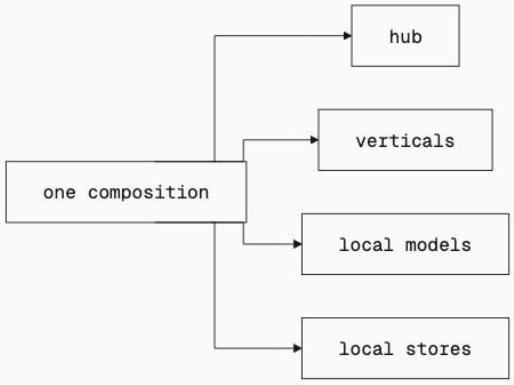
Before the system runs anywhere else, it runs here: one machine, every service beside its neighbours, the full loop from question to answer on hardware you can touch. The local stack is not a demo mode or a trial with features removed. It is the same system arranged for one machine.

You need Docker and about 20 GB of free disk; a local model also wants plenty of memory. Services start side by side, talk over the machine's own network and keep their data in local stores. A developer runs the same composition they ship, and a small firm can run the whole product on one well-chosen computer.

There is no digithings account. Model calls go wherever you point them: a provider with your own key, or Ollama on the machine, one setting apart. Which one you choose decides where your text goes.

## How it works

One Docker Compose file describes which services run, what each needs and how they reach each other. A single command starts the core: identity, the hub, research, retrieval, tracing status, the model router and Ollama. The chat, the vault, the MCP servers and a trace collector are profiles you add. One command stops everything. Sample documents ship with it and load on first run.




one file describes the whole machine; the services inside are the ones that run everywhere

The services behave as they do in any other home, and a request is signed and checked as in a cloud account, because the code never learns where it is running. The only local concessions are friendlier defaults and warm caches.

## Where it fits

The local stack is where the working day happens: development, experiments, demonstrations and small production workloads. The cloud home is the other deployment option, for a team that would rather rent managed infrastructure than keep a machine of its own; nothing in the stack insists on either. The last chapter covers what happens when one machine is no longer enough. Nothing written against the local stack has to be rewritten, so work done locally carries straight over.

17

# Start on your machine

The laptop is usually treated as a developer convenience, a vaguer copy of the real thing. That is backwards. If an application cannot run on your own computer, it is hard to trust or to hand on. So the first deployment of anything built on digithings is the computer in front of you.

## The first hour

Choose the model first: a hosted provider's key, or the setting that keeps every call on Ollama. Decide before you load anything of your own. Then start the stack, load the sample documents and ask about them. The first thing you see is the whole loop: interface, hub, retrieval, a model, a trace.

## The first week

Pick the use case closest to your work and switch on what it needs. For research, the research vertical and retrieval over your notes. For documents, retrieval and the vault. For an agent, the tools you already have, over MCP. Keep the trace on. It shows how the system reached each answer.

## When it outgrows the machine

You will know: a pipeline needing more memory, an index past the disk, more people than one machine should serve. Then move only the module under pressure. Put it on a server you own or in a cloud account, point its neighbours there and carry on.

## Who runs it

Most teams would rather not run Docker, which is what digithings the company is for: the stack set up on your machines or in your cloud account, run and hosted, and the workflows built on top. Write to contact@digithings.ai if that is the help you need.

## If digithings disappears

The code does not need the company. Everything you build keeps running under the MIT licence, and every module ships with architecture notes. Whoever picks it up next, person or agent, will not start from a blank page. The design notes are at digithings.ai/book.